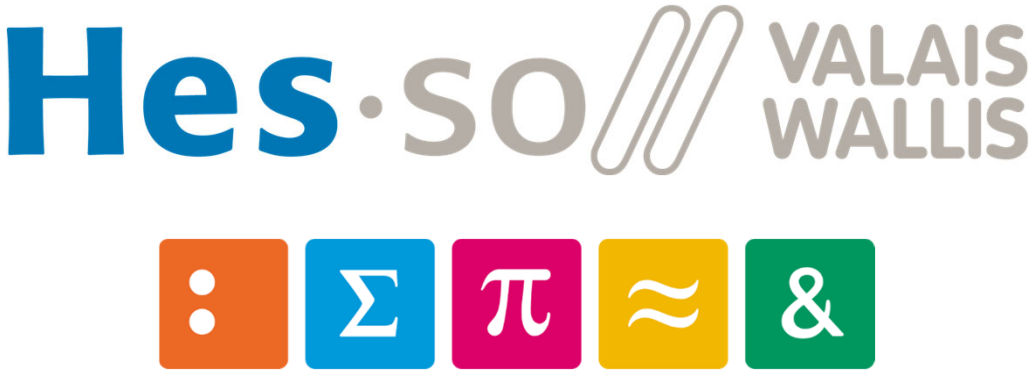


Embedded systems I / embedded programming

‘C’ Programming basics

Pascal Sartoretti

v2.0

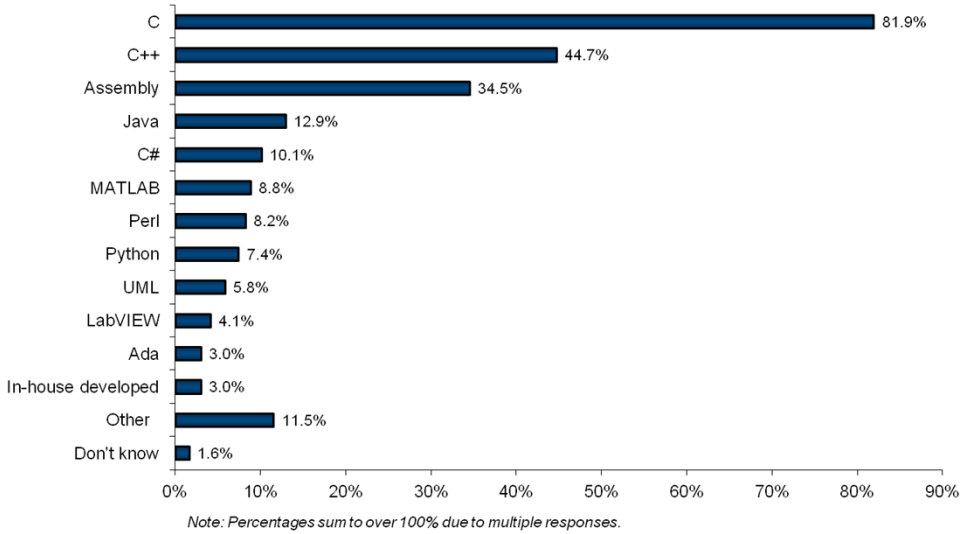
‘C’ : objectives

Be able to program in C language for embedded system

List of chapters

- **C syntax, operators, control statements**
- **Variable types (size)**
- **C functions**
- **Variable scope (local vs global)**
- **Strings, arrays**

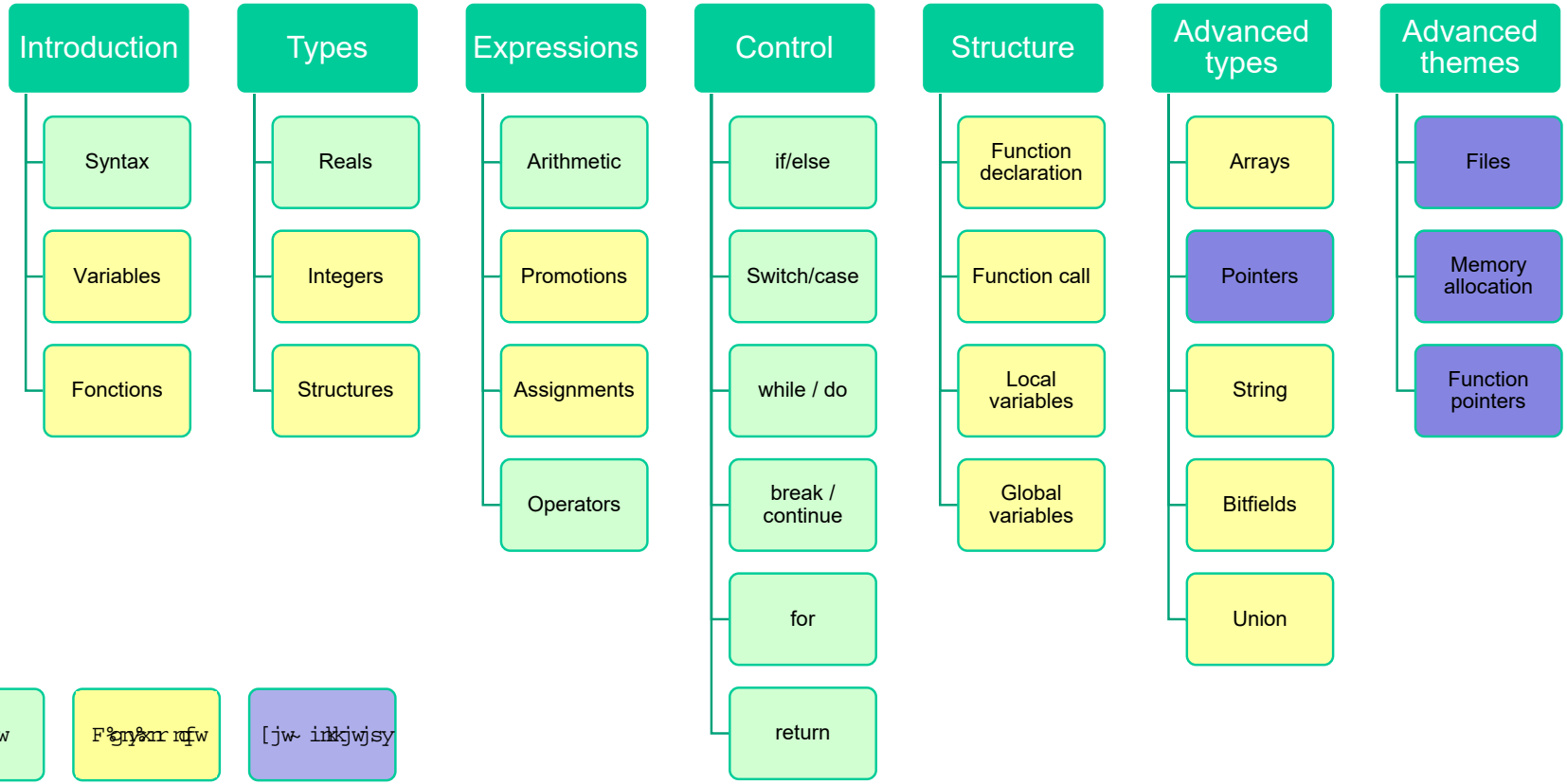
‘C’ : embedded usage information



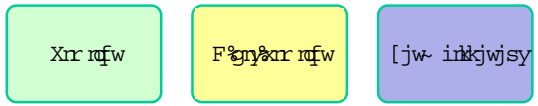
Timeline of C language[11]

Year	Informal name	C Standard
1972	Birth	—
1978	K&R C	—
1989/1990	ANSI C, ISO C	ISO/IEC 9899:1990
1999	C99	ISO/IEC 9899:1999
2011	C11, C1x	ISO/IEC 9899:2011
2018	C17	ISO/IEC 9899:2018
2023*	C23, C2x	TBA

‘C’ : java vs ‘C’

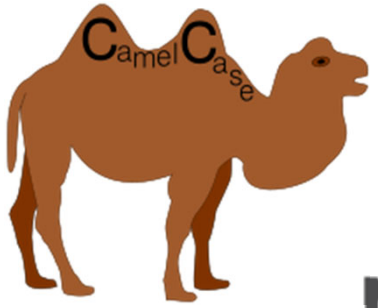


Colors



Syntax, operators, types, control statement in C

‘C’ : CamelCase (in Java)





Naming conventions in Java

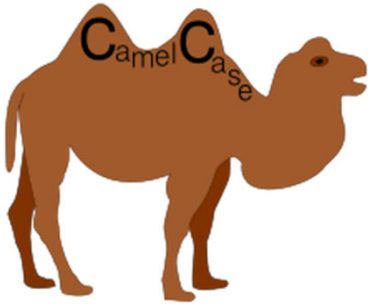
NAMING CONVENTIONS	APPLICATION	EXAMPLES
Lower Camel Case	variables and methods	firstName timeToFirstLoad indexNumber
Upper Camel Case	classes, interfaces, annotations, enums, records	TomcatServer RestController WriteOperation
Screaming Snake Case	constants	INTEREST_RATE MINIMUM_SALARY EXTRA_SAUCE

‘C’ : CamelCase C example

```c
#define FACTOR_NOISE (88.37) // constant
typedef enum
{
  IDLE,
  RUN,
  STOP
}StateMachine;                // type


char timeCounter;             // variable
char addFunc(23,12);          // function
```





‘C’ : constants

```
#define THING 48;                    // a constant
#define NUM_A 88                     // a constant
#define NUM_B 8                      // a constant
#define NUM_C NUM_A - NUM_B          // compiled constant
const char TOTO = 22;                // takes place in flash


 char  val = NUM_C * 2;              // a problem ?
 char  val2 = THING * 2;             // a problem ?
 char  val = NUM_C * 2;              // a problem ?
 TOTO = TOTO * 2;                    // a problem ?
}
```

‘C’ : macros

```
#define SWAP(x,y)\
          int32_t tmp = x; \
          x = y; \
          y = tmp
```

MACRO

```
int32_t  a,b,c;
if(c == 0)
  SWAP(a,b);
```

CALLER CODE

```
int32_t  a,b,c;
if(c == 0)
  int32_t tmp = a;
a = b;
b = tmp;
```

RESULT

‘C’ : macros

```
#define SWAP(x,y)\
          {int32_t tmp = x; \
          X = y; \
          Y = tmp;}
```

MACRO WITH { }

```
 int32_t  a,b,c;
 if(c == 0)
   SWAP(a,b);
 else
   doSomething();
```

CALLER CODE

```
 if(z == 0)
 {
   int32_t tmp = a;
   a = b;
   b = tmp;
 }
;                       // empty statment
else                    // ERROR “parse error before else”
  doSomething();
```

RESULT

‘C’ : simple types

Integer types
- char, int, long
- **Modifiers:** signed, unsigned, short

Floating types
- float, double

Rules
- char <= short int <= int <= long int <= float <= double <= long double

‘C’ : simple types in practice

Type	*Processor architecture*			
	8 bits	*16 bits*	***32 bits***	*64 bits*
char	8 bits	8 bits	**8 bits**	8 bits
short	16 bits	16 bits	**16 bits**	16 bits
int	16 bits	16 bits	**32 bits**	32 bits
long	32 bits	32 bits	**32 bits**	32 or 64 bits
long long	64 bits	64 bits	**64 bits**	64 bits
float	32 bits (24 bits)	32 bits	**32 bits**	32 bits

‘C’ : simple types in practice

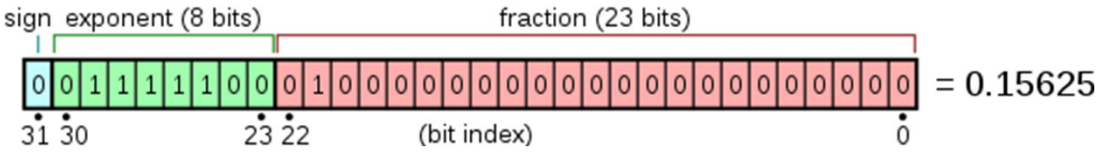
Usage of `#include <stdint.h>`

Type	*Processor architecture*			
	8 bits	*16 bits*	***32 bits***	*64 bits*
uint8_t int8_t	8 bits	8 bits	**8 bits**	8 bits
uint16_t int16_t	16 bits	16 bits	**16 bits**	16 bits
uint32_t int32_t	32 bits	32 bits	**32 bits**	32 bits
uint64_t int64_t	64 bits	64 bits	**64 bits**	64 bits

‘C’ : float type representation

Floating types: (IEE-754)

- **float (SEEEEEEEEFFFF…FFFFF) 32 bits**
- **double (SEEEEEEEEEEEEFFF…FFFF) 64 bits**



$$\text{value} = (-1)^{\text{sign}} \times 2^{(E-127)} \times \left(1 + \sum_{i=1}^{23} b_{23-i} 2^{-i}\right)$$

https://www.h-schmidt.net/FloatConverter/IEEE754.html

‘C’ : caution with float usage

```
float myFloat = 95002174;          // a float variable

 if(myFloat == 95002176)           // test float
 {
   myFloat = 0;                    // do that or not ?
 }
```

```
float volt = 3.2;                  // the float variable
uint32_t mVolt = 3200;             // the integer variable

 volt = volt / 2.0;                // float divide
 mVolt = mVolt / 2;                // integer divide
```

Example above do the same job for the both variables, how choose the best one ?

‘C’ : enum type

Enumerated type is just like an int but with names

```
enum Vals {START, RESTART, STOP};
enum Vals myVal;
```

```
Typedef enum {START, RESTART, STOP}Vals;
Vals myVal;
```

```
Typedef enum {START, RESTART, STOP}Vals;
Vals myVal;
 ...
 myVal = START;              // yes
 myVal = 5;                  // what do you mean ?
 myVal = 70000;              // and here ?
 ...
```

‘C’ : types exercice

What are the valid range for the following values on a STM32 ?

- `uint8_t a`
- `int8_t b`
- `unsigned char c`
- `int d`
- `enum e`

‘C’ : operators syntax

Operators are very similar to Java or Python

Same precedence rules

When not 100% sure, use of () is a good choice

Warning with = and ==

```
warning: suggest parentheses around assignment used as
truth value [-Wparentheses]
204 | if(freq = 4500)
```

priority

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

‘C’ : control statements

```
if()
{
 …
}
else
{
 …
}
```

```
while()
{
 …
}
```

```
do
{
 …
}while();
```

```
for(;;)
{
 …
}
```

```
switch()
{
 case:
 …
 break;
 default:
}
```

```
break;

continue;

return;
```

? (ternary operator)

```
#define MAX(x,y)(((x)>(y))?(x):(y))
```

goto

On error handling only !

functions in C

‘C’ : functions

In C language, all is functions

```
void main(void)
{
   ...
}
```

Embedded C

```
int main(int argc, char * argv[])
{
   ...
}
```

PC program C

‘C’ : functions usage

Prototype (most often in the .h file)

- Arguments (number and types, possibly empty)
- Name of the function
- Return value type

Declaration (in the .c file)

- Defines what the function makes
- Should come after the prototype

Return value

- Is optional, may return void

‘C’ : function prototype

Prototype of a function

myfunctions.h

```
#ifndef _MYFUNCTIONS_H
#define _MYFUNCTIONS_H
#ifdef __cplusplus
extern "C" {
#endif

float average(float a, float b);     // average function
float avg(float, float);             // at least that
#ifdef __cplusplus
}
#endif

#endif                               // end of _MYFUNCTIONS_H
```

‘C’ : function declaration
Declaration of a function

myfunctions.c

```
#include "myfunctions.h"

float average(float a, float b)
{
  return (a+b)/2.0;
}
```

‘C’ : variadic function

Very special, often used with printf

```c
#include <stdarg.h>

int addEmUp(int count, ...)
{
  va_list ap;
  uint32_t sum;
  va_start(ap, count);
  for(uint8_t i=0;i<count;i++)
  {
    sum += va_arg(ap,int);
  }
  va_end(ap);
  return sum;
}
```

```c
tot4 = addEmUp(4,22,55,66,88);                // call it with 4 param example
```

‘C’ : static keyword for functions

- Can’t be used in another file
- Scope limitation

```
static int16_t mult2(uint8_t number)
{
  return (int16_t)number * 2;
}

void main(void)
{
  int16_t a = mult2(150);
}
```

‘C’ : inline keyword for functions

- **Normally, a call to a function requires a jump to this function and return**
- **Inline force to copy this function in place of the call to the function**

```
inline int16_t mult2(uint8_t number)
{
  return (int16_t)number * 2;
}

void main(void)
{
  int16_t a = mult2(150);
}
```

Variable scope

‘C’ : local variable – local scope

- **Exist only during function execution**
- **Value at entry can be undefined**

```
uint16_t countInc(void)
{
  uint16_t counter;                    // not initialized, unknow value
  return counter++;                    // returns unknow + 1
}

uint16_t countInc(void)
{
  uint16_t counter=0;                  // initialized to 0
  return counter++;                    // returns 0 + 1 always, no sense ;-)
}
```

‘C’ : local variable static – local scope

- With static keyword, variable keep alive
- Initialization is made only once (like global)
- The scope is in the function only

```
uint16_t countInc(void)
{
  static uint16_t counter = 0;     // like a global with restricted scope
  return counter++;                // returns old counter + 1
}
```

‘C’ : global variable – file global scope

- **Uninitialized variables are initialized to 0**
- **The scope is to the file**
- **The local same variable name is preceding global**

```
uint8_t counter;                    // global, initialized to 0
uint16_t value = 7;
void countInc(void)
{
  counter++;                        // increments global variable
}

void countInc2(void)
{
  uint8_t counter=0;                // local counter
  counter++;                        // caution, local preceding global
}
```

‘C’ : global variable – multiple files

myvars.h

```
extern uint8_t counter;                    // used for export to other files
```

myvars.c

```
#include "myvars.h"
uint8_t counter=100;                    // global variable declared once only
```

main.c

```
#include "myvars.h"
```

- **Use extern keyword in the .h file**
- **Never place a variable in a .h file !!!!**

STRINGS in C

‘C’ : strings in C, type don’t exist

- **Strings are arrays of bytes**
- **The sign " is used for string, it means null terminated**

```
uint8_t msg1[] = "Hello";          // simple table of 6 bytes (null at end)
char msg2[] = {'H', 'e', 'y', 0}; // simple table of 4 bytes (null at end)
const char msg3[] = "Hi";          // simple const table of 3 bytes
char *msg4 = "Caution";            // caution with that
```

What is where in memory (RAM, Flash) ?

‘C’ : strings manipulation

```
uint8_t msg1[] = "Hello";          // simple table of 6 bytes (null at end)
char msg2[6];                      // simple table of 6 bytes

msg2 = msg1;                       // it’s a dream ;-)
```

You can include <string.h> and have

- `size_t strlen(const char *src)`
- `char * strcpy(char *dst, const char *src)`
- `int strcmp(const char *str1, const char *str2)`
- `strcat, strncmp, memcpy, memmove, ...`

size_t is a long unsigned int for 32 bits processors

‘C’ : strings manipulation, caution

```
uint8_t msg1[] = "Hello";          // simple table of 6 bytes (null at end)
char msg2[] = {'H', 'e', 'y'};     // simple table of 3 bytes
uint8_t msg3[] = "Youhou";         // simple table of 7 bytes (null at end)
```

What is the result of functions below ?

- `strlen(msg1);`
- `strcpy(msg3, msg1);`
- `strcpy(msg1, msg3);`
- `strlen(msg2);`

‘C’ : strings display

You can include <stdio.h> and have

- `int printf(const char *format, ...)`
- `int sprintf(char *dst, const char *format, ...)`
- `scanf, getc, puts, ...`

```
#include <stdio.h>
char text[] = "Hello";                          // simple table
uint8_t var = 23;                               // simple variable

printf("%s", text);                             // string display
printf("Content of var is %d \n", var);         // string display with a value
```

‘C’ : strings escape characters \

Next to \ you can have the escape characters below

- a: bell (speaker)
- b: backspace (non erase)
- f: form feed (clear screen)
- n, r: new line, carriage return
- t, v: tabulator, vertical tabulator
- ‘, “: signle quote, double quote
- \: backslash character
- xnn: hexadecimal character (nn is the value)
- nn: decimal character

```
printf("Hi\bello\\you\x32\n");  // what will be printed
```

‘C’ : strings parameters of %

Next to % you can have

- c: character
- d or i: signed decimal character
- E or e: scientific exponential
- f: floating point value
- s: string of characters
- u: unsigned decimal
- X or x: unsigned hexadecimal
- 0: left pad number of 0
- %: to print %
- +: force positive number to print leading +

```
printf("Number is 0x%04x\n",20);  // what will be printed
```

‘C’ : strings input

scanf is used to read something
It is not often used because difficult to analyze if we type anything else than expected

getc is often preferred, analyze is made after

```
#include <stdio.h>
char str[20];                          // 20 bytes for answer
uint16_t value;                        // simple variable

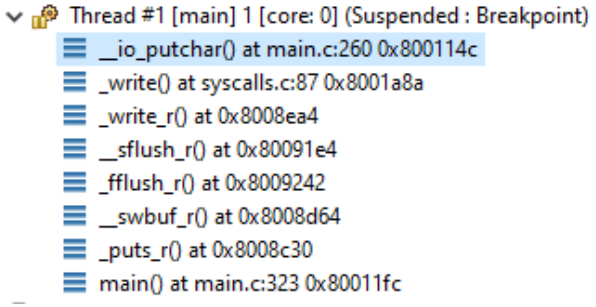
scanf("%d", &value);                   // read decimal and store in value
scanf("%19s", str);                    // read a string
```

‘C’ : strings general input / output

Embedded systems don’t have default I/O like a PC (screen and keyboard)

- printf requests `__io_putchar` or `_write` to be written

printf call stack



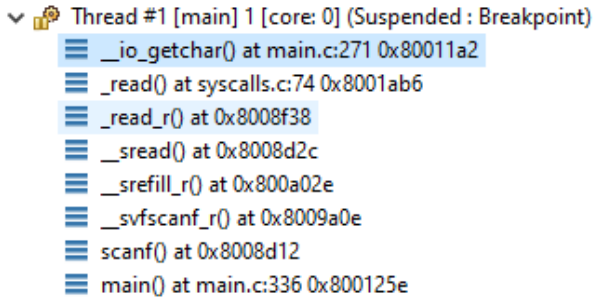

```c
#include <stdio.h>
int __io_putchar(int ch)
{
   ...    // choose what to do with the character to print
}
```

‘C’ : strings general input / output

Embedded systems don’t have default I/O like a PC (screen and keyboard)

- scanf requests `__io_getchar` or `_read` to be written

scanf call stack




```c
#include <stdio.h>
int __io_getchar(void)
{
   ...    // choose how to get the received character
}
```

ARRAYS in C

‘C’ : arrays in C

- Arrays size is fixed and must be know at compile time
- `sizeof()` returns the array size in bytes
- No boundary check !
- Argument passing, caution

```
char arr1[] = {1,2,3,4};          // 4 bytes array initialised
uint16_t arr2[5];                 // 10 bytes array uninitialised
struct MyCar car[10];             // 10 * struct size bytes array
```

‘C’ : arrays in C, passing as argument

```
void arrayPassing(uint8_t array[], uint8_t size)
{
  for(uint8_t i=0; i<size; i++)
  {
    array[i] = array[i] + 10;
  }
}
void main(void)
{
  uint8_t arr[] = {1,2,3,4,5};
  arrayPassing(arr, sizeof(arr));
  printf("%d",arr[0]);
}
```

‘C’ : arrays, multiple dimensions arrays

```
double matrix[3][3];
uint16_t arr2[2][3] = {
    {1,2,3},
    {4,5,6}
  };

uint16_t fnct(uint16_t tbl[3][2][4])
{
  ...
}
```

- As argument in function, size must be given
- Arrays can be initialized when defined

‘C’ : arrays exercices

Write a function who invert a string like below

`stringInvert(uint8_t string[])`

- “pool” -> “loop”

Write a function who swap all words LSB/MSB of a given array: `arrSwap(uint16_t arr[], uint8_t size)`

- Arr[0] = 0x1234 becomes 0x3412
- Arr[1] = 0xABCD becomes 0xCDAB
- …